SEPTEMBER 2024

TFB1013: STRUCTURED PROGRAMMING

FINAL PROJECT PROPOSAL

ELECTRONICS DEVICE PRE-ORDER SYSTEM

LECTURER: DR. AZLINA KAMARUDDIN

FACULTY: FACULTY OF SCIENCES AND INFORMATION TECHNOLOGY

NO.	NAME	STUDENT ID	COURSE
1.	ANTASYA NASUHA BINTI ALFU'AT	22011624	IS
2.	CHE FARIS ISKANDAR BIN CHE AZIZI	22012057	IT
3.	NUR AIN NUHA BINTI HILMI	22011568	IT
4.	NUR HUDA HELMY BINTI MOHD HELMY	22011626	IT
5.	NUR HUSNA SYAZA BINTI ZAINAL ABIDIN	22011693	IT

TABLE OF CONTENT

1.0 Introduction

1.1 Project Overview …..……..... 3

1.2 Objectives of the Project ...……..... 3

2.0 Project Scope

2.1 Project Description …….……..... 5

2.2 Features and Functionality….…….....…… 6

3.0 System Design

3.1 Flowchart ……..…….... 8

3.2 Program Logic Overview …….....…..... 12

4.0 Implementation

4.1 Code Structure ……..... 13

4.2 Key Functions and Modules ……..... 14

4.3 Data Structures Used ……..... 19

4.4 User Interface ……..... 22

5.0 Testing and Results

5.1 Test Plan ……..……..…..... 25

5.2 Sample Outputs ……..... 26

6.0 Conclusion

6.1 Summary of Accomplishments …….....…..... 30

7.0 Appendix

7.1 Full Source Code …….....……... 31

1.0 Introduction

1.1 Project Overview

We propose the development of a novel gadget electronic device pre-order system, which will offer an easy-to-operate interactive platform for customers to make reservations and purchase gadgets about to hit the market. Our system the TechPresto Pre-Order System will offer various models of devices with different features and prices to ensure that customers have a personalized manner of purchasing gadgets. It will be a line-up of gadgets comprising smartphones, tablets, smartwatches, and other related devices with different specifications and configurations to meet the diverse needs and preferences of the target customers. Incorporating intuitive and user-friendly design, TechPresto introduces ease in pre-ordering through detailed product information, options for customization, secure payment gateways, and real-time tracking for orders.

1.2 Objectives of the Project

The main aim of our project is to make this pre-ordering process easy by developing a user-friendly platform on which customers can view, compare, and preorder any electronic devices that would be launched with much ease. Advanced features of this will include device customization, detailed product comparisons, and superior customer service to ensure glitch-free and satisfactory experiences in the pre-ordering process.

We seek to make our platform the first choice among tech enthusiasts when pre-ordering advanced electronic devices by closely listening to the needs of users, offering sophisticated personalization possibilities, and state-of-the-art customer service. This project brings together a range of functions in one smooth, intuitive interface to enable customers to confidently make informed decisions about pre-ordering devices.

Technology is much more than just gadgets; technology is a means for innovation, staying connected, and improving your life. We know the thrill you feel when holding that brand-new gadget in your hands on its first day of release, and so TechPresto has designed an effortless pre-order process where the latest innovations come at your fingertips. Be it the most advanced smartphones, chic tablets, or state-of-the-art smartwatches, we present you with a painstakingly crafted list of premium electronics by top brands.

Our pre-order platform will be designed in the art of ease and efficiency for securing your next device. At the click of a few buttons, one will be able to browse upcoming releases,

compare specifications, and choose exactly the right device that will fit their lifestyle. From high-performance features to options for customization, we will carry a wide array of devices that would interest both the tech enthusiast and the everyday user.

At TechPresto, going beyond the basics means making sure you are one step ahead. We make sure you have exclusive early-bird deals, priority shipping, and add to the perks with extended warranties and premium accessories. Rest assured of safe and timely delivery while having secure pay options for real-time order tracking.

We believe that pre-ordering a product should be as thrilling as the product itself. That's why at TechPresto, we ensure your experience is just as seamless from start to finish with an intuitive interface, ease of navigation, and top-shelf customer service. Whether it's the latest smartphone you are after or a new tablet for work, our system is designed to let you enjoy an unparalleled experience in pre-ordering and bring tomorrow's technology to your doorsteps today.

2.0 Project Scope

2.1 Detailed Description of the Project

TechPresto is a C++ program created to function as an online preorder system for a tech store, allowing users to browse, select, and preorder devices from different categories. The system organizes devices into five main categories: Computing Devices, Communication Devices, Household Appliances, Photography and Imaging Devices, and Healthcare Devices. Within each category, users can explore a list of available devices, filtered by three price ranges: less than 1500 MYR, 1500–5000 MYR, and greater than 5000 MYR. Each device in the catalog provides detailed information, including specifications like name, display or type, processor (when applicable), RAM, storage, price, release date, description, warranty, and accessories. The functions `displayCategories()` and `displayPriceRanges()` make navigation straightforward, while device details are presented through specialized display functions for each category.

For placing orders, users enter personal information such as name, email, address, and payment method (credit, debit, or PayPal). The system validates these inputs to ensure accuracy and confirms the order details using the `verifyOrderDetails()` function before saving them to a file. The `saveOrderDetails()` function writes the confirmed order details to a text file (`order_receipt.txt`), which can later be reviewed by the user for preorder tracking. This feature allows users to verify their order status by checking the saved preorder information, simulating a simple order-tracking system.

The program's structure is built around C++ `structs` to represent each device category, with each struct tailored to the specifications relevant to that category. Display functions such as `displayComputing()`, `displayCommunication()`, and others manage the output of device details, while user prompts, order verification, and order saving are handled by functions like `displayCategories()`, `displayPriceRanges()`, `verifyOrderDetails()`, and `saveOrderDetails()`. To ensure a seamless experience, the program performs input validation for category selection, price range, and personal information, prompting users for correct input where necessary.

In terms of user interaction flow, TechPresto begins by guiding users through category and price range selection, followed by device viewing and order placement. Users finalize the process by entering personal information and checking the order status in `order_receipt.txt`. This system not only provides an organized and accessible browsing experience but also confirms and tracks orders to ensure accuracy.

Future enhancements could include expanding file storage capabilities to accommodate multiple orders in separate files or a database, adding product filtering options for more specific device features, and developing a GUI for a more engaging user experience. Overall, TechPresto effectively combines essential C++ concepts, such as structs, file I/O, validation, and control structures, to create a simplified e-commerce preorder system.

2.2 Features and Functionality

The TechPresto system offers a range of features designed to provide users with a streamlined preorder experience for various tech devices. The system categorizes devices into five main types—Computing Devices, Communication Devices, Household Appliances, Photography and Imaging Devices, and Healthcare Devices. Users can browse devices by category and refine their selection based on price ranges: less than 1500 MYR, 1500 to 5000 MYR, and greater than 5000 MYR. Each category has a dedicated function that displays detailed device information, including name, display/type, processor, RAM, storage, price, release date, description, warranty, and accessories, allowing users to view specifications in a clear, well-structured format.

TechPresto is designed for user-friendly navigation. Functions such as `displayCategories()` and `displayPriceRanges()` guide users through the selection process, helping them make informed choices. After selecting a device, users can place an order by providing personal details like name, email, address, and payment method (credit, debit, or PayPal), with the system validating each input to ensure it meets the required standards. If an invalid entry is detected, such as an empty field or an unsupported payment method, the system prompts users to re-enter the information.

Once the details are entered, the `verifyOrderDetails()` function displays a summary of the order for confirmation. Users are given a final chance to review and verify their details before proceeding, ensuring accuracy in order placement. The `saveOrderDetails()` function then saves the confirmed details to a text file, `order_receipt.txt`, which serves as a persistent record for tracking purposes. Additionally, the program includes an option for users to check their preorder status by viewing the contents of `order_receipt.txt`, which simulates a simple order tracking feature.

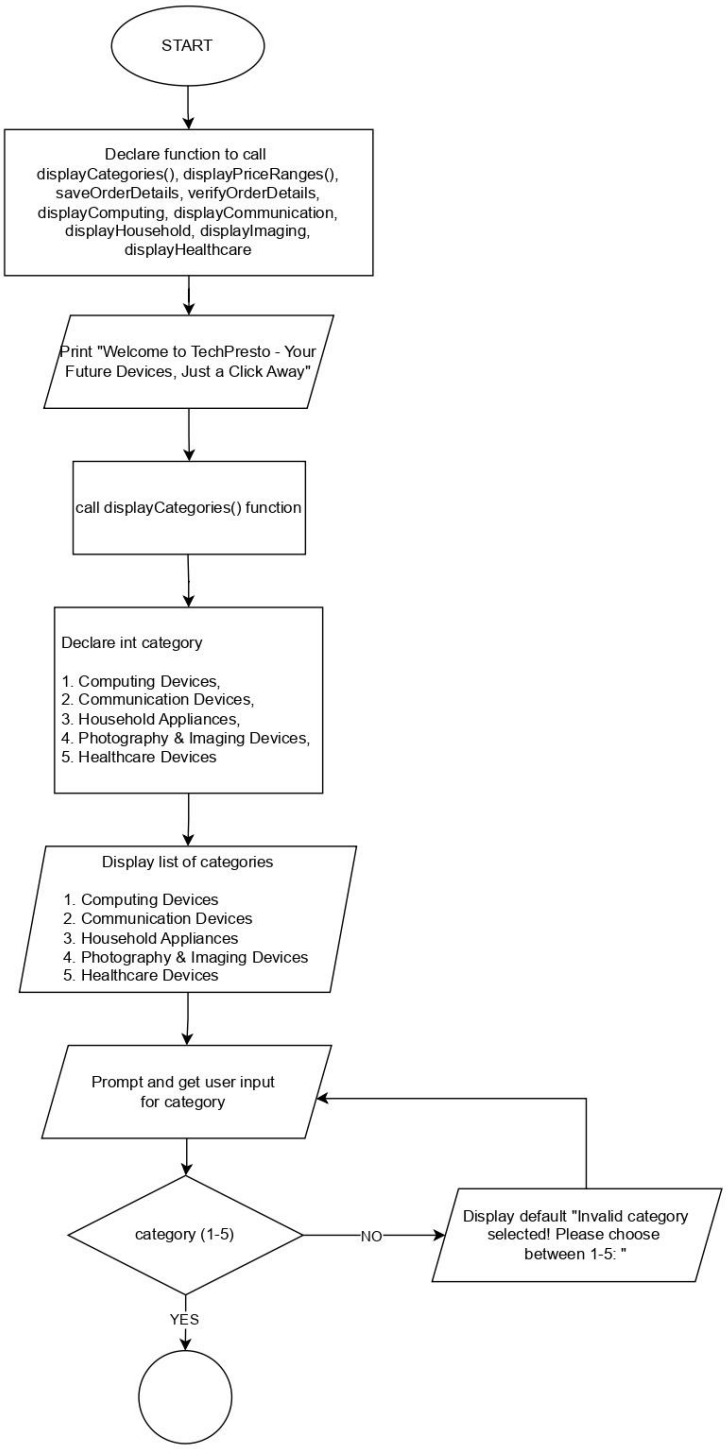
TechPresto incorporates comprehensive error handling to manage potential issues with category and price range selection. If users enter an invalid category or price range, they are

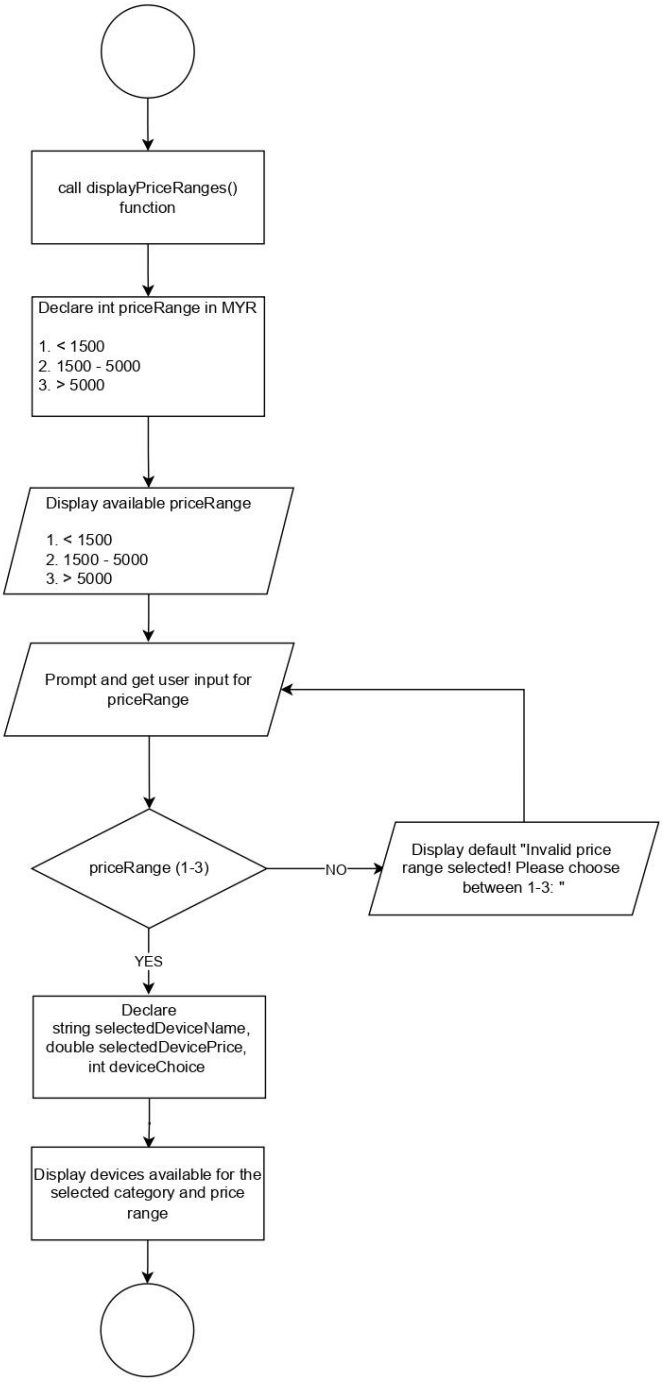
prompted to correct their choice. Invalid inputs are cleared, and the system ignores erroneous data to prevent crashes or unexpected behavior. Interactive prompts enhance the user experience by providing clear instructions, error messages, and confirmations throughout the ordering process.

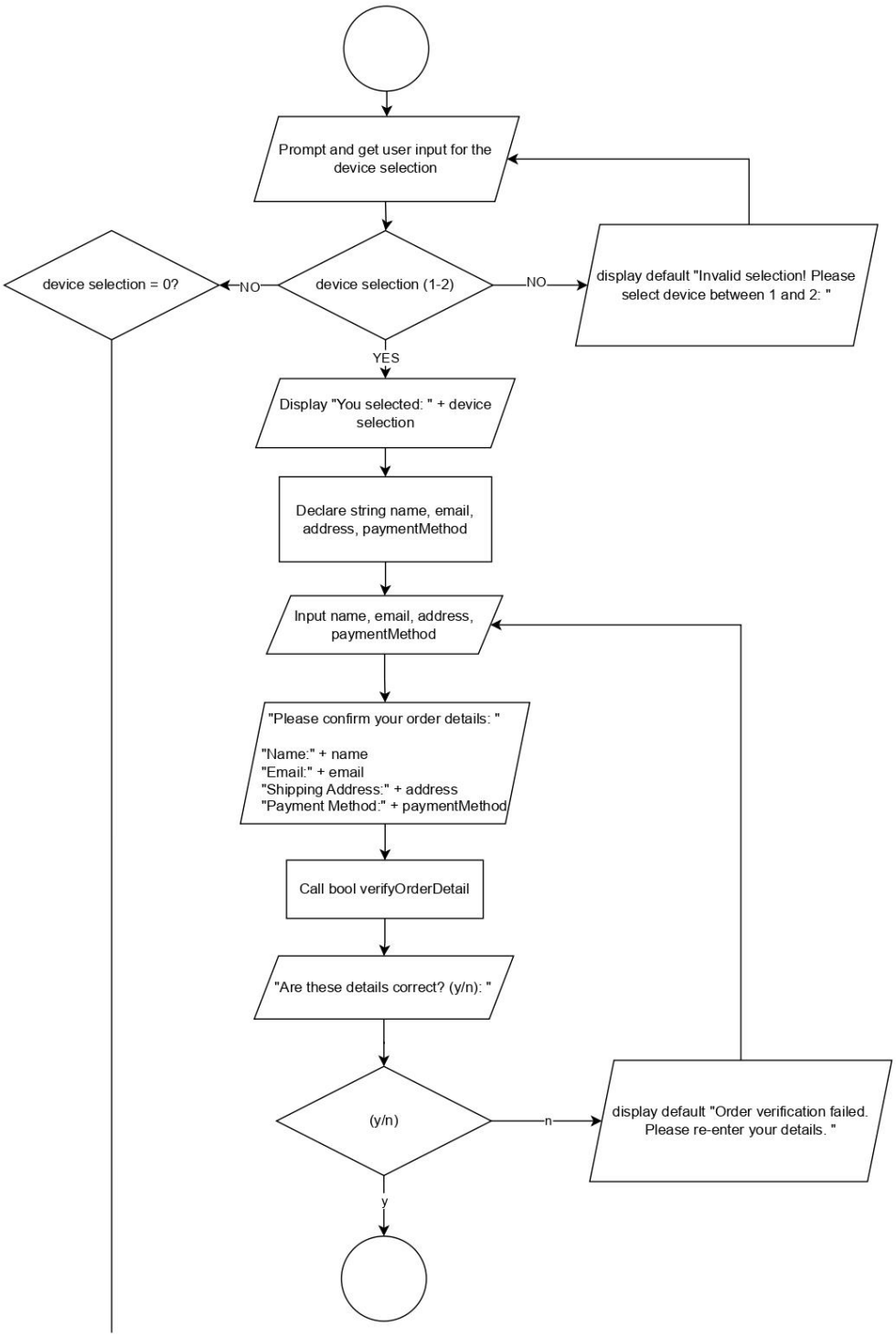
The program also utilizes file I/O for persistent storage, ensuring order details are saved for future reference. This functionality makes TechPresto a robust and practical preorder system, allowing users to browse, select, and purchase devices with accuracy and convenience. Overall, the system demonstrates the use of structured data, file handling, and validation effectively to create a cohesive and user-centric preorder experience.

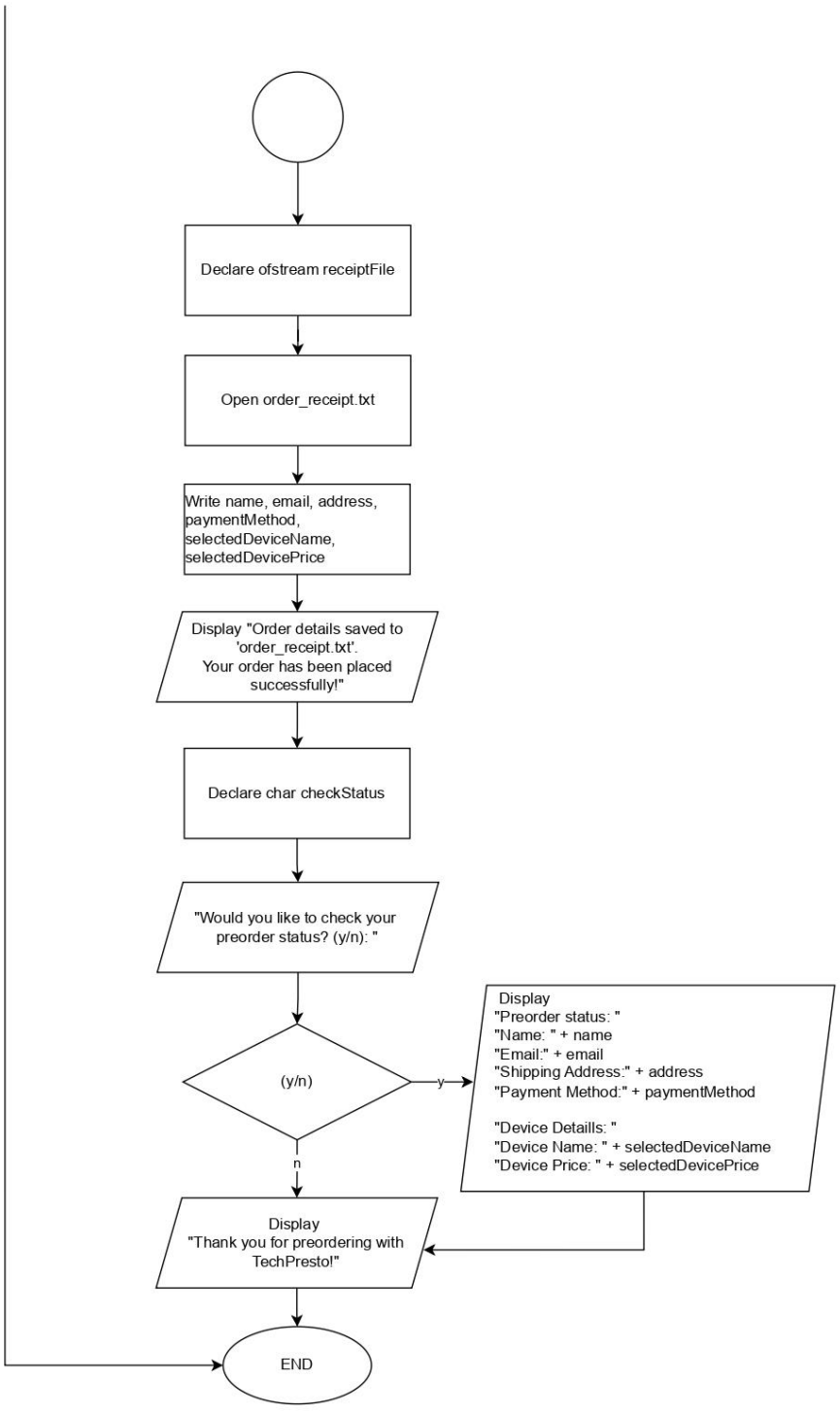
3.0 System Design

3.1 Flowchart




call displayPriceRanges() function
Declare int priceRange in MYR
1. < 1500
2. 1500 - 5000
3. > 5000
Display available priceRange
1. < 1500
2. 1500 - 5000
3. > 5000
Prompt and get user input for priceRange
priceRange (1-3)
NO
Display default "Invalid price range selected! Please choose between 1-3: "
YES
Declare string selectedDeviceName, double selectedDevicePrice, int deviceChoice
Display devices available for the selected category and price range


Prompt and get user input for the device selection
device selection (1-2)
NO
display default "Invalid selection! Please select device between 1 and 2: "
device selection = 0?
NO
YES
Display "You selected: " + device selection
Declare string name, email, address, paymentMethod
Input name, email, address, paymentMethod
"Please confirm your order details: "
"Name:" + name
"Email:" + email
"Shipping Address:" + address
"Payment Method:" + paymentMethod
Call bool verifyOrderDetail
"Are these details correct? (y/n): "
(y/n)
n
display default "Order verification failed. Please re-enter your details. "
y


Declare ofstream receiptFile
Open order_receipt.txt
Write name, email, address, paymentMethod, selectedDeviceName, selectedDevicePrice
Display "Order details saved to 'order_receipt.txt'. Your order has been placed successfully!"
Declare char checkStatus
"Would you like to check your preorder status? (y/n): "
(y/n)
y
n
Display "Preorder status: " "Name: " + name "Email:" + email "Shipping Address:" + address "Payment Method:" + paymentMethod "Device Detaills: " "Device Name: " + selectedDeviceName "Device Price: " + selectedDevicePrice
Display "Thank you for preordering with TechPresto!"
END

3.2 Program Logic Overview

The preorder process begins by welcoming the user to the system, displaying a greeting message and a tagline to set the tone. The user is then presented with a list of device categories, such as Computing Devices, Communication Devices, and Household Appliances. The user selects a category by entering the corresponding number, and the system confirms their choice. Next, the user is prompted to specify their preferred brand and type of device (e.g., Samsung, Laptop). Following this, the system displays predefined price ranges in MYR, and the user selects the desired range.

If the selected brand and device type do not fall within the chosen price range, the system notifies the user and prompts them to reconsider their price selection. Otherwise, the system lists available devices that match the user's preferences, providing detailed information about each option, such as model name, display size, processor type, RAM, storage capacity, price, release date, warranty, and included accessories. The user then selects the device they wish to preorder.

To complete the preorder, the system asks the user to input personal details, including their name, email address, shipping address, and payment method (e.g., PayPal, Credit, or Debit card). Once the details are provided, the system confirms the preorder, displaying a message that includes the user's preorder status, shipping information, and selected payment method. Additionally, the user is given the option to check their order status, where they can see a confirmation that their order has been processed and is on its way. Throughout this process, the system ensures that user input is handled efficiently, with conditional checks for available devices and price ranges, and appropriate prompts to guide the user through a smooth and interactive preorder experience.

4.0 Implementation

4.1 Code Structure

The code structure consists of multiple parts, including the declaration of structs for each device category, helper functions for displaying information, and main logic for user interaction. The following components are included:

- **Header Files**: Includes necessary libraries like `<iostream>`, `<fstream>`, and `<string>` for input-output operations and file handling. It also uses `<vector>` for storing devices and `<limits>` for input validation.

```
#include <iostream>
#include <fstream>
#include <string>
#include <vector>
#include <limits> // For std::numeric_limits
```

- **Device Category Structs**: Each device category (Computing, Communication, Household, Imaging, Healthcare) is represented by a struct containing device attributes such as name, display type, price, and description.
- **Functions**:
  - `displayCategories()` and `displayPriceRanges()` functions show the available options to users for selecting categories and price ranges.
  - `saveOrderDetails()` stores the order information into a file named `order_receipt.txt`.
  - `verifyOrderDetails()` prompts users to confirm their order details before proceeding with the purchase.
  - `displayComputing()`, `displayCommunication()`, `displayHousehold()`, `displayImaging()`, `displayHealthcare()` display detailed information about devices in the selected category.
- **Main Program Flow** `main()`: The main program handles user input, validates it using loops, and controls the flow of the application. It allows users to select categories, price ranges, and devices, and then saves or verifies their order.

4.2 Key Functions and Modules

- **`displayCategories()`**: Displays a list of available device categories. Each category corresponds to a struct type that holds data for various devices.

```
void displayCategories() {
    cout << "Device Categories:\n";
    cout << "1. Computing Devices\n";
    cout << "2. Communication Devices\n";
    cout << "3. Household Appliances\n";
    cout << "4. Photography and Imaging Devices\n";
    cout << "5. Healthcare Devices\n\n";
}
```

- **`displayPriceRanges()`**: Offers three price range options for users to choose from.

```
void displayPriceRanges() {
    cout << "Price Ranges (in MYR):\n";
    cout << "1. < 1500\n";
    cout << "2. 1500 - 5000\n";
    cout << "3. > 5000\n\n";
}
```

- **`saveOrderDetails()`**: Saves the order details, such as the user's name, email, address, payment method, and the selected device's name and price, to a text file for later reference.

```
void saveOrderDetails(const string& name, const string& email, const
string& address, const string& paymentMethod, const string&
deviceName, double devicePrice) {
    ofstream orderFile("order_receipt.txt"); // Append to the file
    if (orderFile.is_open()) {
        orderFile << "Name: " << name << endl;
        orderFile << "Email: " << email << endl;
```

```
        orderFile << "Shipping Address: " << address << endl;
        orderFile << "Payment Method: " << paymentMethod << endl;
        orderFile << "\nDevice Details:\n";
        orderFile << "Device Name: " << deviceName << endl;
        orderFile << "Device Price: " << devicePrice << " MYR\n";
        orderFile << "---------------------------------------\n";
        cout << "Order details saved to 'order_receipt.txt'.\n";
        orderFile.close();
    } else {
        cout << "Unable to open file to save order details.\n";
    }
}
```

- **`verifyOrderDetails()`**: Asks the user to confirm their order details before proceeding to ensure the accuracy of the order.

```
bool verifyOrderDetails(const string& name, const string& email,
const string& address, const string& paymentMethod) {
    cout << "\nPlease verify your order details:\n";
    cout << "Name: " << name << "\n";
    cout << "Email: " << email << "\n";
    cout << "Shipping Address: " << address << "\n";
    cout << "Payment Method: " << paymentMethod << "\n";
    cout << "Are these details correct? (y/n): ";
    char confirmation;
    cin >> confirmation;
    return (confirmation == 'y' || confirmation == 'Y');
}
```

- **Device Display Functions**: Each device category has a corresponding function (`displayComputing`, `displayCommunication`, etc.) that outputs detailed information about the available devices in the selected category.

```
void displayComputing(const Computing &comp) {
```

```
    cout << "Name: " << comp.name << endl;
    cout << "Display: " << comp.display << endl;
    cout << "Processor: " << comp.processor << endl;
    cout << "RAM: " << comp.ram << "GB" << endl;
    cout << "Storage: " << comp.storage << endl;
    cout << "Price: " << comp.price << " MYR" << endl;
    cout << "Release Date: " << comp.releaseDate << endl;
    cout << "Description: " << comp.description << endl;
    cout << "Warranty: " << comp.warranty << endl;
    cout << "Accessories: " << comp.accessories << endl;
    cout << "------------------------" << endl;
}

void displayCommunication(const Communication &comm) {
    cout << "Name: " << comm.name << endl;
    cout << "Display: " << comm.display << endl;
    cout << "Processor: " << comm.processor << endl;
    cout << "RAM: " << comm.ram << "GB" << endl;
    cout << "Storage: " << comm.storage << endl;
    cout << "Price: " << comm.price << " MYR" << endl;
    cout << "Release Date: " << comm.releaseDate << endl;
    cout << "Description: " << comm.description << endl;
    cout << "Warranty: " << comm.warranty << endl;
    cout << "Accessories: " << comm.accessories << endl;
    cout << "------------------------" << endl;
}

void displayHousehold(const Household &house) {
    cout << "Name: " << house.name << endl;
    cout << "Type: " << house.type << endl;
    cout << "Capacity: " << house.capacity << "Liters" << endl;
    cout << "Features: " << house.features << endl;
    cout << "Price: " << house.price << " MYR" << endl;
```

```
    cout << "Release Date: " << house.releaseDate << endl;
    cout << "Description: " << house.description << endl;
    cout << "Warranty: " << house.warranty << endl;
    cout << "Accessories: " << house.accessories << endl;
    cout << "------------------------" << endl;
}

void displayImaging(const Imaging &img) {
    cout << "Name: " << img.name << endl;
    cout << "Type: " << img.type << endl;
    cout << "Resolution: " << img.resolution << endl;
    cout << "Price: " << img.price << " MYR" << endl;
    cout << "Release Date: " << img.releaseDate << endl;
    cout << "Description: " << img.description << endl;
    cout << "Warranty: " << img.warranty << endl;
    cout << "Accessories: " << img.accessories << endl;
    cout << "------------------------" << endl;
}

void displayHealthcare(const Healthcare &health) {
    cout << "Name: " << health.name << endl;
    cout << "Type: " << health.type << endl;
    cout << "Features: " << health.features << endl;
    cout << "Price: " << health.price << " MYR" << endl;
    cout << "Release Date: " << health.releaseDate << endl;
    cout << "Description: " << health.description << endl;
    cout << "Warranty: " << health.warranty << endl;
    cout << "Accessories: " << health.accessories << endl;
    cout << "------------------------" << endl;
}
```

- **`cin.fail()`**: The program ensures that invalid inputs (like a non-existent category or price range) are handled gracefully by using loops and checking the input type.

```
// Validate category selection
    while (cin.fail() || category < 1 || category > 5) {
        cin.clear(); // Clear the error state
        cin.ignore(numeric_limits<streamsize>::max(), '\n'); //
Ignore invalid input
        cout << "Invalid category selected! Please choose between 1-
5: ";
        cin >> category;
    }
```

```
// Validate price range selection
    while (cin.fail() || priceRange < 1 || priceRange > 3) {
        cin.clear(); // Clear the error state
        cin.ignore(numeric_limits<streamsize>::max(), '\n'); //
Ignore invalid input
        cout << "Invalid price range selected! Please choose between
1-3: ";
        cin >> priceRange;
    }
```

```
// Validate device selection
        while (cin.fail() || deviceChoice < 0 || deviceChoice >
devices.size()) {
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            cout << "Invalid selection! Please select a device between 1-" <<
devices.size() << " or 0 to exit: ";
            cin >> deviceChoice;
        }
```

4.3 Data Structures Used

- **Structs**: The core data structures used in the program are structs for each device category (Computing, Communication, Household, Imaging, Healthcare). These structs contain device attributes like name, display size, price, release date, warranty, and other relevant details.

```
// Structs for different device categories
struct Computing {
    string name;
    string display;
    string processor;
    int ram; // in GB
    string storage;
    double price; // in MYR
    string releaseDate;
    string description;
    string warranty;
    string accessories;
};

struct Communication {
    string name;
    string display;
    string processor;
    int ram; // in GB
```

```
    string storage;
    double price; // in MYR
    string releaseDate;
    string description;
    string warranty;
    string accessories;
};

struct Household {
    string name;
    string type;
    double capacity; // in Liters
    string features;
    double price; // in MYR
    string releaseDate;
    string description;
    string warranty;
    string accessories;
};

struct Imaging {
    string name;
    string type;
    string resolution;
    double price; // in MYR
    string releaseDate;
    string description;
    string warranty;
    string accessories;
};

struct Healthcare {
    string name;
```

```
    string type;
    string features;
    double price; // in MYR
    string releaseDate;
    string description;
    string warranty;
    string accessories;
};
```

- **Arrays/Vectors** `<vector>`: The devices in each category are stored in vectors, allowing dynamic storage and manipulation. The vectors are populated based on the user-selected price range.

```
if (category == 1) { // Computing Devices
        vector<Computing> devices;
```

```
} else if (category == 2) { // Communication Devices
        vector<Communication> devices;
```

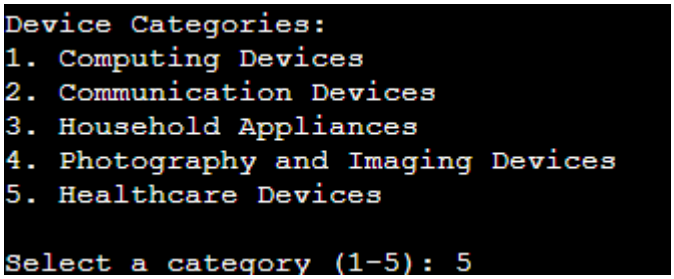
```
} else if (category == 3) { // Household Appliances
        vector<Household> devices;
```

```
} else if (category == 4) { // Imaging Devices
        vector<Imaging> devices;
```

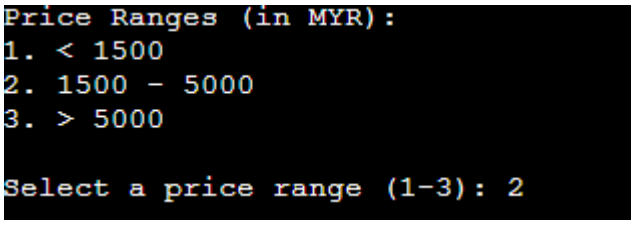
```
} else if (category == 5) { // Healthcare Devices
        vector<Healthcare> devices;
```

4.4 User Interface

1. **Category Selection**: The user selects a device category (Computing, Communication, etc.) from a menu.



2. **Price Range Selection**: The user selects a price range (< 1500 MYR, 1500 - 5000 MYR, > 5000 MYR).



3. **Device Selection**: Based on the chosen category and price range, the available devices are displayed. The user selects one for preorder.
   e.g.: category 5, price range 2

```
Available Healthcare Devices:
1.
Name: Fitbit Charge 5
Type: Fitness Tracker
Features: Built-in GPS, Stress Management
Price: 900 MYR
Release Date: 2023-08-15
Description: Advanced fitness tracker with health metrics.
Warranty: 1-year standard warranty
Accessories: Charging cable
------------------------
```

```
2.
Name: Withings Body+ Smart Scale
Type: Smart Scale
Features: Body Composition, Bluetooth
Price: 400 MYR
Release Date: 2023-05-01
Description: Smart scale that syncs with your phone.
Warranty: 1-year standard warranty
Accessories: User manual
------------------------

Select a device to preorder (1-2) or 0 to exit: 1
You selected: Fitbit Charge 5
```

4. **Order Details**: After selecting a device, the user is prompted to enter their order details (name, email, address, payment method).

```
Enter your name: husna
Enter your email: husna@gmail.com
Enter your address: kd
Enter payment method (credit/debit/paypal): credit
```

5. **Order Confirmation**: The user is given a chance to verify their order before confirming the details.

```
Please verify your order details:
Name: husna
Email: husna@gmail.com
Shipping Address: kd
Payment Method: credit
Are these details correct? (y/n): y
Order details saved to 'order_receipt.txt'.
Your order has been placed successfully!
```

6. **File Output**: The order details are saved to a file (`order_receipt.txt`), providing a record of the transaction.

main.cpp | order_receipt.txt

```
1  Name: husna
2  Email: husna@gmail.com
3  Shipping Address: kd
4  Payment Method: credit
5
6  Device Details:
7  Device Name: Fitbit Charge 5
8  Device Price: 900 MYR
9  ----------------------------------------
```

5.0 Testing and Results

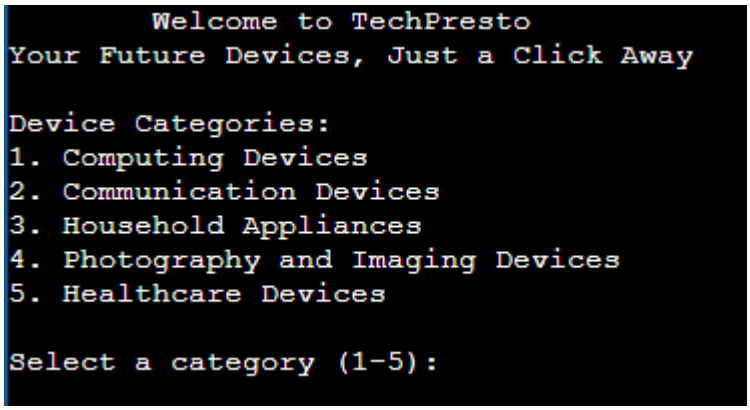
This section will outline the testing procedures and display sample outputs generated during the execution of the program.

5.1 Test Plan

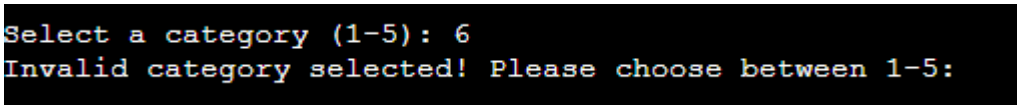
Description	Expected Result	Actual Result	Status
Launch program and view main categories	Program displays available categories with IDs 1 to 5	Program displays categories correctly	Pass
Select an invalid category (e.g., 6)	Program displays an error and prompts for a valid category	Program displays error message and prompts correctly	Pass
Choose a valid category and view price ranges	Program displays price range options: "< 1500", "1500 - 5000", "> 5000"	Program displays price range options correctly	Pass
Select an invalid price range (e.g., 0)	Program displays an error and prompts for a valid price range	Program displays error message and prompts correctly	Pass
Select a valid price range and view device options	Program displays devices within the chosen category and price range	Program displays devices correctly based on category and price range	Pass
Choose an invalid device selection (e.g., out-of-range number)	Program displays an error and prompts for a valid selection	Program displays error message and prompts correctly	Pass
Enter user details for order (name, email, address, payment method)	Program accepts and verifies user details	Program accepts and verifies user details correctly	Pass
Confirm order with valid details and save to file	Program saves order details to `order_receipt.txt`	Program saves details to file correctly and confirms to the user	Pass
Cancel order confirmation (e.g., press 'n' for verification)	Program returns to main menu without saving order	Program does not save details and returns to main menu	Pass
Repeat program operation for multiple orders	Program continues operation and returns to the main menu after each complete order	Program correctly loops back to main menu after each order	Pass
Exit program	Program stops execution gracefully	Program exits successfully	Pass

5.2 Sample Outputs

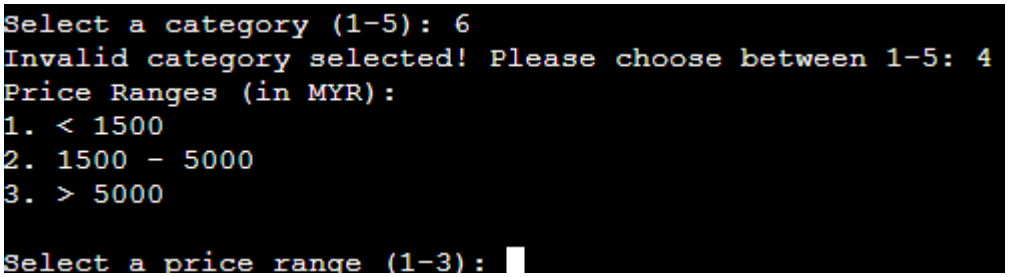
1. Displaying Categories:



2. Invalid Category Selection:



3. Displaying Price Ranges:



4. Device Details Display (Example for Photography and Imaging Devices):

```
Select a price range (1-3): 3

Available Imaging Devices:
1.
Name: Canon EOS R5
Type: Mirrorless Camera
Resolution: 45 MP
Price: 10000 MYR
Release Date: 2023-09-01
Description: Professional camera with 8K video recording.
Warranty: 2-year extended warranty
Accessories: Camera bag, Lens
------------------------
2.
Name: Nikon Z9
Type: Mirrorless Camera
Resolution: 45.7 MP
Price: 12000 MYR
Release Date: 2024-01-15
Description: Flagship camera with advanced features.
Warranty: 2-year extended warranty
Accessories: Extra battery
------------------------

Select a device to preorder (1-2) or to 0 exit:
```

5. Order Confirmation and Verification:

```
You selected: Canon EOS R5
```

```
Please verify your order details:
Name: tasa
Email: tasasusila@gmail.com
Shipping Address: johor riau
Payment Method: debit
Are these details correct? (y/n): y
Order details saved to 'order_receipt.txt'.
Your order has been placed successfully!
```

6. Exiting Program:

```
Preorder Status:
Name: fr
Email: df
Shipping Address: fdar
Payment Method: credit

Device Details:
Device Name: Samsung Smart Oven MC32K7055KT
Device Price: 2000 MYR
----------------------------------------

Thank you for preordering with TechPresto!
```

```
Select a device to preorder (1-2) or to 0 exit: 0
Exiting program...
```

6.0 Conclusion

The C++ program designed for TechPresto serves as a functional electronic device preorder system. It effectively integrates various C++ concepts, including functions, arrays, loops, selections, and file handling, to create an interactive and user-friendly application. The program allows users to select from a variety of devices across multiple categories, such as Computing, Communication, Household, Imaging, and Healthcare. Each device is displayed with detailed information like specifications, price, and warranty. The inclusion of a price range filter and category selection provides a personalized shopping experience. Additionally, the user can verify their order details and save the final order in a file, offering a smooth process for managing preorders.

By implementing the order saving feature with file handling, the program ensures that user data is persistently stored in a text file (`order_receipt.txt`), creating a record of transactions. The validation of user input is a key aspect of the program's reliability, preventing invalid choices and ensuring that users select the correct options. The program's structure allows for easy expansion, enabling more device categories, features, or pricing models to be added in the future. Overall, the program showcases a solid grasp of C++ principles and provides a functional tool for managing electronic device preorders.

6.1 Summary of Accomplishments

In this project, several key accomplishments were achieved:

1. **Device Catalog with Categories and Filters:**
    a. The program effectively categorizes devices into Computing, Communication, Household, Imaging, and Healthcare categories.
    b. It allows users to filter devices based on price range, providing a more tailored shopping experience.
2. **Comprehensive Data Display:**
    a. Devices are displayed with detailed attributes, including specifications, price, release date, description, warranty, and accessories, giving users enough information to make an informed decision.
3. **User Input Validation:**
    a. The program ensures valid input for category selection, price range, and device choice, preventing errors and providing a smoother user experience.
4. **File Handling for Order Management:**
    a. Users' order details are saved to a text file (`order_receipt.txt`), ensuring that each transaction is recorded for future reference.
5. **Interactive Menu and Loop:**
    a. The program utilizes a menu-driven interface that allows users to navigate through different tasks easily.
    b. The main loop ensures that users can perform multiple operations without restarting the program, providing flexibility.
6. **Use of C++ Features:**
    a. The program successfully applies various C++ features such as arrays (for storing device information), functions (to modularize tasks), loops (for repetitive actions), and selections (for decision-making processes), demonstrating the versatility of C++ for building real-world applications.

7.0 Appendix

7.1 Full Source Code

```
//Electronic Device Preorder System

#include <iostream>
#include <fstream>
#include <string>
#include <vector>
#include <limits> // For std::numeric_limits
using namespace std;

// Structs for different device categories
struct Computing {
    string name;
    string display;
    string processor;
    int ram; // in GB
    string storage;
    double price; // in MYR
    string releaseDate;
    string description;
    string warranty;
    string accessories;
};

struct Communication {
    string name;
    string display;
    string processor;
    int ram; // in GB
    string storage;
    double price; // in MYR
    string releaseDate;
    string description;
    string warranty;
    string accessories;
};

struct Household {
    string name;
    string type;
    double capacity; // in Liters
    string features;
    double price; // in MYR
    string releaseDate;
```

```
    string description;
    string warranty;
    string accessories;
};

struct Imaging {
    string name;
    string type;
    string resolution;
    double price; // in MYR
    string releaseDate;
    string description;
    string warranty;
    string accessories;
};

struct Healthcare {
    string name;
    string type;
    string features;
    double price; // in MYR
    string releaseDate;
    string description;
    string warranty;
    string accessories;
};

void displayCategories() {
    cout << "Device Categories:\n";
    cout << "1. Computing Devices\n";
    cout << "2. Communication Devices\n";
    cout << "3. Household Appliances\n";
    cout << "4. Photography and Imaging Devices\n";
    cout << "5. Healthcare Devices\n\n";
}

void displayPriceRanges() {
    cout << "Price Ranges (in MYR):\n";
    cout << "1. < 1500\n";
    cout << "2. 1500 - 5000\n";
    cout << "3. > 5000\n\n";
}

void saveOrderDetails(const string& name, const string& email, const string&
address, const string& paymentMethod, const string& deviceName, double
devicePrice) {
    ofstream orderFile("order_receipt.txt"); // Append to the file
```

```
    if (orderFile.is_open()) {
        orderFile << "Name: " << name << endl;
        orderFile << "Email: " << email << endl;
        orderFile << "Shipping Address: " << address << endl;
        orderFile << "Payment Method: " << paymentMethod << endl;
        orderFile << "\nDevice Details:\n";
        orderFile << "Device Name: " << deviceName << endl;
        orderFile << "Device Price: " << devicePrice << " MYR\n";
        orderFile << "---------------------------------------\n";
        cout << "Order details saved to 'order_receipt.txt'.\n";
        orderFile.close();
    } else {
        cout << "Unable to open file to save order details.\n";
    }
}

bool verifyOrderDetails(const string& name, const string& email, const string&
address, const string& paymentMethod) {
    cout << "\nPlease verify your order details:\n";
    cout << "Name: " << name << "\n";
    cout << "Email: " << email << "\n";
    cout << "Shipping Address: " << address << "\n";
    cout << "Payment Method: " << paymentMethod << "\n";
    cout << "Are these details correct? (y/n): ";
    char confirmation;
    cin >> confirmation;
    return (confirmation == 'y' || confirmation == 'Y');
}

void displayComputing(const Computing &comp) {
    cout << "Name: " << comp.name << endl;
    cout << "Display: " << comp.display << endl;
    cout << "Processor: " << comp.processor << endl;
    cout << "RAM: " << comp.ram << "GB" << endl;
    cout << "Storage: " << comp.storage << endl;
    cout << "Price: " << comp.price << " MYR" << endl;
    cout << "Release Date: " << comp.releaseDate << endl;
    cout << "Description: " << comp.description << endl;
    cout << "Warranty: " << comp.warranty << endl;
    cout << "Accessories: " << comp.accessories << endl;
    cout << "------------------------" << endl;
}

void displayCommunication(const Communication &comm) {
    cout << "Name: " << comm.name << endl;
    cout << "Display: " << comm.display << endl;
    cout << "Processor: " << comm.processor << endl;
```

```
    cout << "RAM: " << comm.ram << "GB" << endl;
    cout << "Storage: " << comm.storage << endl;
    cout << "Price: " << comm.price << " MYR" << endl;
    cout << "Release Date: " << comm.releaseDate << endl;
    cout << "Description: " << comm.description << endl;
    cout << "Warranty: " << comm.warranty << endl;
    cout << "Accessories: " << comm.accessories << endl;
    cout << "------------------------" << endl;
}

void displayHousehold(const Household &house) {
    cout << "Name: " << house.name << endl;
    cout << "Type: " << house.type << endl;
    cout << "Capacity: " << house.capacity << "Liters" << endl;
    cout << "Features: " << house.features << endl;
    cout << "Price: " << house.price << " MYR" << endl;
    cout << "Release Date: " << house.releaseDate << endl;
    cout << "Description: " << house.description << endl;
    cout << "Warranty: " << house.warranty << endl;
    cout << "Accessories: " << house.accessories << endl;
    cout << "------------------------" << endl;
}

void displayImaging(const Imaging &img) {
    cout << "Name: " << img.name << endl;
    cout << "Type: " << img.type << endl;
    cout << "Resolution: " << img.resolution << endl;
    cout << "Price: " << img.price << " MYR" << endl;
    cout << "Release Date: " << img.releaseDate << endl;
    cout << "Description: " << img.description << endl;
    cout << "Warranty: " << img.warranty << endl;
    cout << "Accessories: " << img.accessories << endl;
    cout << "------------------------" << endl;
}

void displayHealthcare(const Healthcare &health) {
    cout << "Name: " << health.name << endl;
    cout << "Type: " << health.type << endl;
    cout << "Features: " << health.features << endl;
    cout << "Price: " << health.price << " MYR" << endl;
    cout << "Release Date: " << health.releaseDate << endl;
    cout << "Description: " << health.description << endl;
    cout << "Warranty: " << health.warranty << endl;
    cout << "Accessories: " << health.accessories << endl;
    cout << "------------------------" << endl;
}
```

```
int main() {
    cout << "\tWelcome to TechPresto\n"<<"Your Future Devices, Just a Click
Away\n\n";

    displayCategories();
    int category;
    cout << "Select a category (1-5): ";
    cin >> category;

    // Validate category selection
    while (cin.fail() || category < 1 || category > 5) {
        cin.clear(); // Clear the error state
        cin.ignore(numeric_limits<streamsize>::max(), '\n'); // Ignore invalid
input
        cout << "Invalid category selected! Please choose between 1-5: ";
        cin >> category;
    }



    displayPriceRanges();
    int priceRange;
    cout << "Select a price range (1-3): ";
    cin >> priceRange;

    // Validate price range selection
    while (cin.fail() || priceRange < 1 || priceRange > 3) {
        cin.clear(); // Clear the error state
        cin.ignore(numeric_limits<streamsize>::max(), '\n'); // Ignore invalid
input
        cout << "Invalid price range selected! Please choose between 1-3: ";
        cin >> priceRange;
    }

    // Display devices and validate device selection
    int deviceChoice;
    string selectedDeviceName;
    double selectedDevicePrice;

    if (category == 1) { // Computing Devices
        vector<Computing> devices;

        if (priceRange == 1) {
            devices = {
```

```
                {"ASUS VivoBook 12", "11.6-inch HD", "Intel Celeron N3350", 4,
"64GB eMMC", 1200, "2023-05-01", "Affordable and compact laptop for basic
tasks.", "1-year standard warranty", "Power adapter"},
                {"Lenovo IdeaPad 1", "14-inch HD", "AMD A6-9220e", 4, "64GB
eMMC", 1400, "2023-07-10", "Lightweight laptop for everyday use.", "1-year
standard warranty", "Protective sleeve"}
            };
        } else if (priceRange == 2) {
            devices = {
                {"Dell Inspiron 15 3000", "15.6-inch Full HD", "Intel Core i3
11th Gen", 8, "256GB SSD", 2800, "2024-02-15", "Reliable laptop for work and
entertainment.", "1-year standard warranty", "External mouse, Laptop bag"},
                {"Acer Aspire 5", "15.6-inch Full HD", "AMD Ryzen 5 5500U", 8,
"512GB SSD", 3500, "2023-11-20", "Budget-friendly performance laptop.", "1-year
standard warranty", "USB-C charger"}
            };
        } else if (priceRange == 3) {
            devices = {
                {"Microsoft Surface Laptop Studio", "14.4-inch PixelSense
Flow", "Intel Core i7 11th Gen", 16, "512GB SSD", 7800, "2023-09-05", "High-
performance laptop for creators.", "2-year extended warranty", "Surface Pen,
Carry case"},
                {"HP Spectre x360 16", "16-inch 4K OLED", "Intel Core i7 13th
Gen", 16, "1TB SSD", 9000, "2024-03-10", "Premium 2-in-1 laptop with stunning
display.", "2-year extended warranty", "Active pen, External USB-C hub"}
            };
        }

        cout << "\nAvailable Devices:\n";
        for (int i = 0; i < devices.size(); ++i) {
            cout << i + 1 << ".\n";
            displayComputing(devices[i]);
        }
        cout << "\nSelect a device to preorder (1-" << devices.size() << ") or
0 to exit: ";
        cin >> deviceChoice;

        if (deviceChoice == 0) {
            cout << "Exiting program...\n";
            return 0 ; // Exits the function (if not in main(), replace with
appropriate handling)
        }

        // Validate device selection
        while (cin.fail() || deviceChoice < 0 || deviceChoice > devices.size())
{
            cin.clear();
```

```
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            cout << "Invalid selection! Please select a device between 1-" <<
devices.size() << " or 0 to exit: ";
            cin >> deviceChoice;
        }

        // Store selected device details for the receipt
        selectedDeviceName = devices[deviceChoice - 1].name;
        selectedDevicePrice = devices[deviceChoice - 1].price;

        cout << "You selected: " << selectedDeviceName << "\n";

    } else if (category == 2) { // Communication Devices
        vector<Communication> devices;

        if (priceRange == 1) {
            devices = {
                {"Xiaomi Redmi Note 11", "6.43-inch FHD+ AMOLED", "Snapdragon
680", 4, "64GB", 1000, "2023-06-10", "Affordable smartphone with high-resolution
display.", "1-year standard warranty", "USB-C charger, Protective case"},
                {"Realme C33", "6.5-inch HD+", "Unisoc T612", 3, "32GB", 800,
"2023-03-15", "Budget smartphone with excellent battery life.", "1-year standard
warranty", "Screen protector, Charger"}
            };
        } else if (priceRange == 2) {
            devices = {
                {"Samsung Galaxy A54", "6.4-inch Super AMOLED", "Exynos 1380",
8, "128GB", 2200, "2024-01-05", "Mid-range phone with advanced camera
capabilities.", "1-year standard warranty", "Fast charger, Silicon case"},
                {"OnePlus Nord CE 3 Lite", "6.72-inch FHD+ LCD", "Snapdragon
695", 8, "128GB", 1800, "2023-09-12", "Smooth-performing phone with 120Hz
display.", "1-year standard warranty", "USB-C charger, Transparent case"}
            };
        } else if (priceRange == 3) {
            devices = {
                {"iPhone 15 Pro Max", "6.7-inch Super Retina XDR OLED", "A17
Pro", 8, "256GB", 6500, "2023-09-22", "Premium smartphone with top-notch
performance.", "1-year standard warranty", "MagSafe charger, Screen
protector"},
                {"Samsung Galaxy Z Fold5", "7.6-inch Foldable Dynamic AMOLED",
"Snapdragon 8+ Gen 2", 12, "512GB", 8800, "2023-08-01", "Foldable phone with
advanced multitasking capabilities.", "2-year extended warranty", "S Pen, Case"}
            };
        }

        cout << "\nAvailable Communication Devices:\n";
        for (int i = 0; i < devices.size(); ++i) {
```

```
            cout << i + 1 << ".\n";
            displayCommunication(devices[i]);
        }
        cout << "\nSelect a device to preorder (1-" << devices.size() << ") or
0 to exit: ";
        cin >> deviceChoice;

        if (deviceChoice == 0) {
            cout << "Exiting program...\n";
            return 0 ; // Exits the function
        }

        while (cin.fail() || deviceChoice < 0 || deviceChoice > devices.size())
{
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            cout << "Invalid selection! Please select a device between 1-" <<
devices.size() << " or 0 to exit: ";
            cin >> deviceChoice;
        }

        // Store selected device details for the receipt
        selectedDeviceName = devices[deviceChoice - 1].name;
        selectedDevicePrice = devices[deviceChoice - 1].price;

        cout << "You selected: " << selectedDeviceName << "\n";

    } else if (category == 3) { // Household Appliances
        vector<Household> devices;

        if (priceRange == 1) {
            devices = {
                {"Sharp ESX710 Washing Machine", "Top Load", 7, "Quick wash,
Eco mode", 1000, "2023-03-01", "Compact washing machine ideal for small
households.", "1-year standard warranty", "Detergent dispenser"},
                {"Midea Air Fryer MF-TN35A", "Air Fryer", 3.5, "Adjustable
temperature, Timer", 450, "2023-04-10", "Compact and efficient air fryer for
quick meals.", "1-year standard warranty", "Fryer basket"}
            };
        } else if (priceRange == 2) {
            devices = {
                {"Samsung Smart Oven MC32K7055KT", "Microwave with Grill", 32,
"Hot Blast™, Slim Fry™", 2000, "2024-01-20", "Versatile oven with multiple
cooking options.", "1-year standard warranty", "Grill rack, Crusty plate"},
                {"Dyson Pure Cool Air Purifier TP04", "Tower Fan & Air
Purifier", 0, "HEPA filter, Real-time air quality monitoring", 3500, "2023-08-
```

```
15", "Efficient air purifier with cooling fan feature.", "2-year extended
warranty", "Remote control"}
            };
        } else if (priceRange == 3) {
            devices = {
                {"LG Side by Side Refrigerator", "Refrigerator", 600, "Inverter
compressor, Smart diagnosis", 4500, "2024-03-15", "Spacious refrigerator with
energy efficiency.", "2-year extended warranty", "Ice tray"},
                {"Bosch Built-in Coffee Machine", "Coffee Maker", 0, "OneTouch
function, Auto milk frother", 6000, "2023-10-05", "Premium coffee machine for
coffee enthusiasts.", "2-year extended warranty", "Water filter"}
            };
        }

        cout << "\nAvailable Household Appliances:\n";
        for (int i = 0; i < devices.size(); ++i) {
            cout << i + 1 << ".\n";
            displayHousehold(devices[i]);
        }
        cout << "\nSelect a device to preorder (1-" << devices.size() << ") or
0 to exit: ";
        cin >> deviceChoice;

        if (deviceChoice == 0) {
            cout << "Exiting program...\n";
            return 0 ; // Exits the function
        }

        while (cin.fail() || deviceChoice < 0 || deviceChoice > devices.size())
{
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            cout << "Invalid selection! Please select a device between 1-" <<
devices.size() << " or 0 to exit: ";
            cin >> deviceChoice;
        }

        // Store selected device details for the receipt
        selectedDeviceName = devices[deviceChoice - 1].name;
        selectedDevicePrice = devices[deviceChoice - 1].price;

        cout << "You selected: " << selectedDeviceName << "\n";

    } else if (category == 4) { // Imaging Devices
        vector<Imaging> devices;
```

```
        if (priceRange == 1) {
            devices = {
                {"Canon EOS 2000D", "DSLR Camera", "18 MP", 1500, "2023-05-01",
"Entry-level DSLR for budding photographers.", "1-year standard warranty",
"Camera bag"},
                {"Nikon Coolpix B500", "Point and Shoot Camera", "16 MP", 800,
"2023-06-15", "User-friendly camera with zoom capabilities.", "1-year standard
warranty", "Batteries"}
            };
        } else if (priceRange == 2) {
            devices = {
                {"Sony Alpha a6400", "Mirrorless Camera", "24.2 MP", 3500,
"2023-11-20", "Compact mirrorless camera with fast autofocus.", "2-year extended
warranty", "Lens kit"},
                {"Fujifilm X-T30", "Mirrorless Camera", "26.1 MP", 4500, "2023-
01-10", "Versatile mirrorless camera for professional use.", "2-year extended
warranty", "Tripod"}
            };
        } else if (priceRange == 3) {
            devices = {
                {"Canon EOS R5", "Mirrorless Camera", "45 MP", 10000, "2023-09-
01", "Professional camera with 8K video recording.", "2-year extended warranty",
"Camera bag, Lens"},
                {"Nikon Z9", "Mirrorless Camera", "45.7 MP", 12000, "2024-01-
15", "Flagship camera with advanced features.", "2-year extended warranty",
"Extra battery"}
            };
        }

        cout << "\nAvailable Imaging Devices:\n";
        for (int i = 0; i < devices.size(); ++i) {
            cout << i + 1 << ".\n";
            displayImaging(devices[i]);
        }
        cout << "\nSelect a device to preorder (1-" << devices.size() << ") or
to 0 exit: ";
        cin >> deviceChoice;

        if (deviceChoice == 0) {
            cout << "Exiting program...\n";
            return 0 ; // Exits the function
        }

        while (cin.fail() || deviceChoice < 0 || deviceChoice > devices.size())
{
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
```

```
            cout << "Invalid selection! Please select a device between 1-" <<
devices.size() << " or 0 to exit: ";
            cin >> deviceChoice;
        }

        // Store selected device details for the receipt
        selectedDeviceName = devices[deviceChoice - 1].name;
        selectedDevicePrice = devices[deviceChoice - 1].price;

        cout << "You selected: " << selectedDeviceName << "\n";

    } else if (category == 5) { // Healthcare Devices
        vector<Healthcare> devices;

        if (priceRange == 1) {
            devices = {
                {"Xiaomi Mi Band 5", "Fitness Tracker", "Heart Rate Monitor,
Sleep Tracking", 200, "2023-02-10", "Affordable fitness tracker with essential
features.", "1-year standard warranty", "Charging cable"},
                {"Philips Sonicare ProtectiveClean 6100", "Electric
Toothbrush", "Pressure Sensor, Smart Timer", 300, "2023-03-05", "Electric
toothbrush for superior dental care.", "1-year standard warranty", "Travel
case"}
            };
        } else if (priceRange == 2) {
            devices = {
                {"Fitbit Charge 5", "Fitness Tracker", "Built-in GPS, Stress
Management", 900, "2023-08-15", "Advanced fitness tracker with health metrics.",
"1-year standard warranty", "Charging cable"},
                {"Withings Body+ Smart Scale", "Smart Scale", "Body
Composition, Bluetooth", 400, "2023-05-01", "Smart scale that syncs with your
phone.", "1-year standard warranty", "User manual"}
            };
        } else if (priceRange == 3) {
            devices = {
                {"Apple Watch Series 8", "Smartwatch", "ECG, Blood Oxygen
Monitoring", 1800, "2023-09-22", "Versatile smartwatch with health tracking.",
"2-year extended warranty", "Sport band"},
                {"Oura Ring Generation 3", "Smart Ring", "Sleep Tracking,
Activity Monitoring", 1000, "2024-03-10", "Compact ring for health insights.",
"2-year extended warranty", "Charging dock"}
            };
        }

        cout << "\nAvailable Healthcare Devices:\n";
        for (int i = 0; i < devices.size(); ++i) {
            cout << i + 1 << ".\n";
```

```
            displayHealthcare(devices[i]);
        }
        cout << "\nSelect a device to preorder (1-" << devices.size() << ") or
0 to exit: ";
        cin >> deviceChoice;

        if (deviceChoice == 0) {
            cout << "Exiting program...\n";
            return 0 ; // Exits the function
        }

        while (cin.fail() || deviceChoice < 0 || deviceChoice > devices.size())
{
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            cout << "Invalid selection! Please select a device between 1-" <<
devices.size() << " or 0 to exit: ";
            cin >> deviceChoice;
        }

        // Store selected device details for the receipt
        selectedDeviceName = devices[deviceChoice - 1].name;
        selectedDevicePrice = devices[deviceChoice - 1].price;

        cout << "You selected: " << selectedDeviceName << "\n";
    }

    // Order details
    string name, email, address, paymentMethod;

    while (true) { // Loop until valid input is received
        cout << "\nEnter your name: ";
        cin.ignore(); // Ignore any leftover newline character
        getline(cin, name);

        // Check for null input
        if (name.empty()) {
            cout << "Name cannot be empty. Please re-enter.\n";
            continue; // Prompt for name again
        }

        cout << "Enter your email: ";
        getline(cin, email);

        // Check for null input
        if (email.empty()) {
```

```
            cout << "Email cannot be empty. Please re-enter.\n";
            continue; // Prompt for email again
        }

        cout << "Enter your address: ";
        getline(cin, address);

        // Check for null input
        if (address.empty()) {
            cout << "Address cannot be empty. Please re-enter.\n";
            continue; // Prompt for address again
        }

        cout << "Enter payment method (credit/debit/paypal): ";
        getline(cin, paymentMethod);

        // Check for null input and specific values
        if (paymentMethod.empty() ||
            (paymentMethod != "credit" && paymentMethod != "debit" &&
paymentMethod != "paypal")) {
            cout << "Payment method must be 'credit', 'debit', or 'paypal'.
Please re-enter.\n";
            continue; // Prompt for payment method again
        }

        // Verify and save order details
        if (verifyOrderDetails(name, email, address, paymentMethod)) {
            saveOrderDetails(name, email, address, paymentMethod,
selectedDeviceName, selectedDevicePrice);
            cout << "Your order has been placed successfully!\n";
            break; // Exit the loop upon successful order placement
        } else {
            cout << "Order verification failed. Please re-enter your
details.\n";
        }
    }

    char checkStatus;
    cout << "\nWould you like to check your preorder status? (y/n): ";
    cin >> checkStatus;

    if (checkStatus == 'y' || checkStatus == 'Y') {
        ifstream orderFile("order_receipt.txt");
        if (orderFile.is_open()) {
            string line;
            cout << "\nPreorder Status:\n";
```

```
            while (getline(orderFile, line)) {
                cout << line << endl;
            }
            cout << "\nThank you for preordering with TechPresto!\n";
            orderFile.close();
        } else {
            cout << "No preorder details found.\n";
        }
    } else {
        cout << "Thank you for preordering with TechPresto!\n";
    }

    return 0;
}
```